

Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- XQuery Nedir?
 - İlişkisel veritabanlarında SQL sorgu dili kullanılırken, XML tipinde dosyalar üzerinde sorgulama yapmak için XQuery kullanılır
 - XQuery, XML dosyaları üzerinde verileri çekmek, listelemek, sıralamak ve koşullu sorgular yazmak için kullanılan bir sorgulama dilidir
 - Web servisleri ve XML tabanlı veri değiştirme süreçleri için daha kullanışlıdır
- XML Dosya Yapısı
 - XML dosyaları hiyerarşik (ağaç yapısı) bir yapıya sahiptir
 - Her tag'bir element olarak adlandırılır
 - En tepedeki elemente "root element" denir (ör. <company>, <bookstore>)
 - Elementler kendi aralarında parent-child (ebeveyn-çocuk) ilişkisine sahiptir
 - Aynı seviyedeki elementler birbirine sibling (kardeş) olarak adlandırılır
- Attribute (Öznitelik) Kullanımı
 - Bir tag'in yanında ekstra bilgi vermek istendiğinde attribute olarak tanımlanır
 - Attribute bir element olarak değil, tag'in içine yazılan bir değer olarak gelir
 - Attribute'un primary key gibi bir kısıtı yoktur; aynı attribute değeri farklı kayıtlarda tekrarlanabilir
 - Örnek: SSN, Dnumber gibi alanlar attribute olarak tanımlanmıştır
- XQuery Sözdizimi ve İfadeler
 - XQuery'de 5 temel ifade bulunmaktadır (FLOWER - "çiçek" okunuşuyla):
 - * **for**: Döngü ile satır satır okuma yapar (SQL'deki SELECT INTO mantığına benzer)
 - * **let**: Değişken tanımlamak için kullanılır
 - * **where**: Koşul belirtmek için kullanılır (SQL'deki WHERE ile aynı mantık)
 - * **order by**: Sonuçları sıralamak için kullanılır
 - * **return**: Sonuç döndürmek için kullanılır
 - Değişken tanımlarken başına \$ işareti konur (ör. \$emp)
 - Yorum satırı için (: :) içine yazılır

Hocanın Özellikle Vurguladığı Kısımlar

- XML Yapısında Parent-Child-Sibling İlişkileri
 - Her elementin altında child'ları vardır, bunlar arasında sibling ilişkisi bulunur
 - Bu hiyerarşik yapıyı anlamak XQuery sorgularını yazmak için temeldir
- Attribute ve Element Farkı
 - Bir veri element olarak tanımlanabileceği gibi attribute olarak da tanımlanabilir
 - Primary key mantığı attribute'larda geçerli değildir; aynı değer tekrarlanabilir
 - Hangi durumda hangisinin tercih edileceği tasarım kararına bağlıdır
- XQuery'de Eşlemeler (Joins)
 - Birden fazla tablo kullanıldığında (employee, department, works_on, dependent) ve-riler önce değişkenlere atanır
 - Değişkenlere atanan veriler arasındaki eşlemeler where koşulunda gerçekleştirilir
 - SQL'deki JOIN işlemine karşılık gelir ancak sözdizimi farklıdır
- BasicX Programı
 - XQuery sorgularını çalıştırmak için "Basics" adlı program kullanılır

- Programın editor penceresine sorgu yazılır, çalıştır butonu ile sonuçlar görüntülenir
- XML dosyasının konumu ile dosya kaydetme konumunun aynı olması gerekir
- Dosya yolu belirtilerek XML kaynağı program tarafından yüklenir

Kısa Tekrar Notları

- XQuery, XML için SQL gibidir; verileri çekmek, listelemek, sıralamak için kullanılır
- XML'de // işareti ile herhangi bir hiyerarşi seviyesinde doğrudan elemente ulaşılabilir
- Tek / işareti ile sıralı olarak aşağı inilir (hiyerarşik yolda ilerleme)
- Attribute'lara ulaşırken başına @ işareti konur
- * (yıldız) işareti ile bir elementin tüm child'larına ulaşılabilir
- data() fonksiyonu ile tag arasındaki veriyi (text-only) elde edilir, tag'ler olmadan sadece veri gösterilir
- matches() fonksiyonu ile regex benzeri eşleşme yapılabilir
- XQuery'de for döngüsü ile elemanlar tek tek taranır; \$x her elemanı temsil eder
- Birden fazla tablo kullanıldığında her tablodan gerekli alanlar önce değişkenlere atanır, sonra where ile eşlenir

Detaylı Açıklamalar

- **XQuery ve XML İlişkisi:** Ders boyunca şirket veritabanı (company database) XML formatında ele alınmıştır. Employee, department, works_on, dependent ve dept_locations tablolarının XML karşılıkları incelenmiştir. Her tabloda alanlar ya element olarak (tag içinde) ya da attribute olarak (tag yanında @ ile) tanımlanmıştır. Hangi alanın hangi yolda tanımlanacağına ilişkin kesin bir kural olmamakla birlikte, primary key benzeri alanların attribute olarak tanımlanması yaygın bir uygulamadır.
- **XQuery Sorgularında Temel Mantık:** İlk olarak for \$x in doc('company.xml')/company/employee şeklinde basit bir employee listeleme sorgusu gösterilmiştir. Bu sorgu, company.xml dosyasındaki company root elementinin altındaki tüm employee elementlerini sırayla tarar ve her birini \$x değişkenine atar. return kısmında \$x/fname yazıldığında sadece ad bilgisi, return \$x yazıldığında tüm employee kaydı listelenir.
- **Koşullu Sorgular (where):** matches(\$x/address, 'Seattle') ifadesi ile Seattle'da yaşayan çalışanlar bulunmuştur. SQL'deki LIKE '%Seattle%' ifadesine karşılık gelir. İki tablo kullanımda ise where \$emp/ssn = \$w/essn eşlemesi ile JOIN işlemi gerçekleştirilmiştir.
- **Birden Fazla Tablo Kullanımı:** Örneğin, “Franklin Wong’un çalıştığı projelerin numaralarını bul” sorusunda employee ve works_on tablolarından veriler çekilmiştir. Önce let ile \$company := doc(...) tanımlaması yapılmış, ardından her tablodan gerekli alanlar ayrı değişkenlere atanmıştır. where koşulunda hem isim filtresi (\$fname = 'Franklin' ve \$lname = 'Wong') hem de SSN eşlemesi (\$ssn = \$essn) uygulanmıştır.
- **Sıralama (order by):** “Satış departmanının hangi şehirlerde ofisi var?” sorusunda order by kullanılarak sonuçların alfabetik sıralanması sağlanmıştır. order by \$dlog ifadesi ile dept_locations değerlerine göre sıralama yapılmıştır.
- **Üç Tablo ile Karmaşık Sorgu:** “Elizabeth isimli akrabası olan çalışanın yöneticisinin adını bul” sorusunda dependent, employee (çalışan) ve employee (yönetici) olmak üzere üç tablo kullanılmıştır. Çalışanın süper_visor_ssn alanı ile yöneticinin SSN'i eşlenerek yöneticiye ulaşılmıştır. Bu sorgu, bir employee tablosunun kendi içinde join işlemine benzemektedir.
- **Kesişim (Intersect) Uygulaması:** Hem “Operating System projesinde çalışan” hem de “Software departmanında çalışan” kişilerin bulunması istendiğinde, iki ayrı sorgu yazılmış ve bu sorgular where koşullarında eşlenerek kesişim oluşturulmuştur. SQL'deki INTERSECT operatörüne karşılık gelmektedir.
- **Not:** İsterseniz bu dersin alt yazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.